PHASE 1 — FOUNDATION · CHAPTER 06

oauth2-proxy

OIDC Authentication Middleware

One security checkpoint at the door — every service inherits authentication from a single, centrally managed OIDC middleware.



oauth2-proxy



HAProxy



Nginx



Authentik OIDC

Why oauth2-proxy Exists

Deploying OIDC authentication separately in every service is repetitive, error-prone, and hard to audit — each integration is a potential weak link.

- GitLab
- Vault
- SonarQube
- Harbor
- *+ 8 more services*



The Security Checkpoint

Think of oauth2-proxy as a security checkpoint at the building entrance. Instead of each office checking ID cards, one guard at the door checks everyone.

If the credential is valid, the guard stamps your hand and lets you in. The offices never see the credential — only the stamped hand.

1 CHECKPOINT replaces N separate OIDC integrations — every backend trusts one centralized identity check instead of implementing its own.

The Authentication Flow



1. User requests a protected service through HAProxy.
2. oauth2-proxy finds no valid session and redirects to Authentik.
3. User logs in; Authentik returns an auth code, then tokens.
4. oauth2-proxy validates the token and creates a session cookie.
5. Request is forwarded to Nginx/backend with X-Remote-User headers.

Backends never see credentials — only a trusted, header-stamped identity.

Standing Up the Service

1 · System User

Create a dedicated system account with no login shell to run oauth2-proxy under least privilege.

```
useradd --system --shell nologin
oauth2-proxy
```

2 · Install Binary

Download the pre-compiled v7.6.0 release, extract, and place it on the path as an executable.

```
wget oauth2-proxy-v7.6.0.linux-
amd64.tar.gz
```

3 · Session Secret

Generate a strong random 32-byte value to encrypt session cookies and store it securely.

```
secrets.token_urlsafe(32)
```

Core Config File

oauth2-proxy.cfg controls networking, session behavior, and the OIDC provider connection to Authentik.



Network

- `http_address = 127.0.0.1:4180 (localhost only)`
- `reverse_proxy = true — trusts X-Forwarded-* from HAProxy`



Session

- `cookie_httponly = true, cookie_samesite = Lax`
- `cookie_expire = 72h, cookie_refresh = 1h`



OIDC Provider

- `provider = oidc, provider_display_name = Authentik SSO`
- `oidc_issuer_url → Authentik; redirect_url → /oauth2/callback`

Authorization & Header Passthrough

Protect Everything by Default

All paths require authentication except an explicit allow-list:

```
skip_auth_routes = [
  /healthz (GET),
  /oauth2/callback (GET)
]
```

- `preferred_username_claim = "email"`
- `skip_jwt_bearer_token = false (signature checked)`

Identity Headers to Backend

Once validated, oauth2-proxy stamps the request with identity headers the backend can trust:

```
X-Auth-Request-User: {user}
```

```
X-Auth-Request-Email: {email}
```

```
X-Auth-Request-Groups: {groups}
```

```
Remote-User: {user}
```

pass_access_token = false — the raw access token is never forwarded downstream.

Credentials, Routing & Go-Live

OIDC Credentials

Get Client ID/Secret from Authentik: Applications → GitLab → Provider.

Route via HAProxy

backend be_oauth2_proxy → server 127.0.0.1:4180, health-checked on /healthz.

Start the Service

systemctl enable --now oauth2-proxy, then verify with systemctl status.

```
oauth2-proxy.service — Active (running)

Main PID: 9999 (oauth2-proxy)  ·  listening on 127.0.0.1:4180
```

Testing the Complete Auth Flow

1. Health Check

GET /healthz (unauthenticated) → 200 OK

2. Protected Route

GET / without cookie → 302 redirect to Authentik

3. Browser Login

Log in as john → consent → session cookie set

4. Session Cookie

Set-Cookie: oauth2_session=… present on success

5. Header Passthrough

Backend logs show X-Remote-User: john

Success looks like: browser redirected to Authentik login, credentials accepted, and the user lands back on the original path with a valid session — no backend saw a password.

Fine-Tuning Access Control



JWKS Token Validation

Public keys fetched automatically from Authentik's .well-known/openid-configuration endpoint.



Group-Based Authorization

allowed_groups = ["users"] restricts access to specific LDAP/OIDC groups (requires groups scope).



Email Domain Restriction

email_domains = ["devsecops.com"] limits sign-in to a specific organizational domain.



Session Refresh

cookie_refresh = 1h re-validates the token hourly without forcing the user to log in again.

Monitoring & Security Considerations



Monitoring & Logging

- journalctl -xeu oauth2-proxy -f — live login attempts & token results
- logging_level = "debug" reveals header injection & routing decisions
- HAProxy /stats endpoint shows backend health for oauth2-01



Security Considerations

- Token Validation — JWTs from Authentik are cryptographically verified
- Session Protection — cookies are HttpOnly (+ Secure once TLS is on)
- CSRF Protection — SameSite=Lax blocks cross-site request forgery
- Token Expiration — sessions expire after 72h regardless of cookie

Common Problems & Fixes

Redirect loop on login	*Session cookie not set/recognized*	Check oauth2-proxy logs for token errors; verify OIDC credentials & redirect URI.
"invalid client" error	*Client ID/Secret mismatch with Authentik*	Copy exact values from Authentik provider config; restart oauth2-proxy.
JWKS / token validation fails	*oauth2-proxy can't reach Authentik*	curl the .well-known/openid-configuration endpoint; check Authentik logs.
HAProxy shows backend DOWN	*oauth2-proxy not listening on 4180*	Check systemctl status & netstat -tulnp; inspect journalctl for errors.

Chapter 06 Validation Script

validate-chapter-06.sh runs 8 automated checks against sec-edge-01 to confirm the deployment is production-ready:

- Service active
- Listening :4180
- Config file exists
- Auth headers configured
- Health check responds
- Protected endpoint redirects
- HAProxy routes correctly
- Logs present

```
Passed: 8   Failed: 0

"Phase 1 Foundation Complete! All chapters 01-06 validated."
```

What We Built & Why It Matters

What We Built	Purpose
oauth2-proxy service	Enforce OIDC auth before accessing any service
Session management	Cached sessions prevent re-auth on every request
Request header injection	Pass authenticated user info to backends
OIDC integration	Seamless connection with Authentik
Health check bypass	Allow monitoring endpoints without auth



Key Takeaway — The Authentication Checkpoint

oauth2-proxy unifies DNS, LDAP, Authentik, HAProxy and Nginx into one security model — no backend needs to implement authentication; every service inherits it from the centralized checkpoint.

PHASE 1 — FOUNDATION

Complete!

- DNS — split-brain, all services resolvable
- Users — LDAP directory with 3+ test users
- Identity — Authentik OIDC provider
- Edge Proxy — HAProxy TLS termination + routing
- Web Gateway — Nginx URL rewriting + proxying
- Authentication — oauth2-proxy OIDC middleware

NEXT STEPS

- Test full OIDC flow in browser at https://10.0.2.12/
- Log in with john / JohnPassword123!
- Verify access to backend services
- Phase 2: deploy GitLab, Harbor, and more